

# 3300

## Speaker 1: Mobile OS & Architecture Philosophy

核心任务： 负责开场，定下基调。不要只做“报幕员”，你要抛出第一个冲突点：桌面 OS (Windows) 和 移动 OS 的本质区别，以及 Android 与 iOS 的哲学之争。

时长目标： 4-5 分钟PPT 页数： 5-6 页

### Slide 1: Title Page (封面)

- 页面视觉：
  - 大标题： **Beyond the Desktop: Mobile & Embedded Operating Systems**
  - 副标题： *Architecture, Challenges, and Convergence*
  - 成员：

| name         | matric No. |
|--------------|------------|
| YUE CHENGHAO | 227154     |
| HUA JIE      |            |
| WANG KAILUN  |            |
  - 背景图建议： 左边是一部被拆解的智能手机，右边是一个机械臂或汽车仪表盘，中间有虚线连接。
- 演讲要点 (Script Idea):
  - "Good morning/afternoon everyone. When we think of an 'Operating System', we usually think of Windows or macOS on a laptop. But today, we are going to talk about the **invisible** rulers of our world."
  - "From the phone in your pocket (Mobile OS) to the brakes in your car (Embedded OS). We are Group [X], and I am [Name]."
- 注意点 (High Score Tips):
  - 眼神交流 (Eye Contact): 开场不要看屏幕！站在讲台中央，微笑着环视全班。这是拿“Confidence”分数的关键时刻。
  - 声音 (Voice): 第一句话声音要洪亮，抓住大家的注意力。

## Slide 2: The Core Conflict (核心冲突：为什么要非传统OS?)

- 页面视觉：
  - 左边：一台插着电源的台式机（图标：High Power, Mouse/Keyboard, Unlimited Energy）。
  - 右边：一部手机（图标：Battery Limit, Touch Screen, Unstable Network）。
  - 文字：**Constraints define Architecture (限制决定架构)**。
- 核心内容 (Key Points):
  - 引用资料：移动 OS 必须在严苛的功耗限制（TDP）和电池约束下运行 ^^。
- 演讲要点 (Script Idea):
  - "So, why do we need a 'Non-Conventional' OS? Why can't we just put Windows 11 on a phone?"
  - "It comes down to **Constraints**. A desktop has unlimited power from the wall and a mouse for precision."
  - "But a mobile device? It runs on a battery^^, and it uses a fat finger for input. These constraints force Mobile OS to be designed completely differently."
- 注意点:
  - 手势 (Gesture): 说 "Desktop" 时指向一边，说 "Mobile" 时举起自己的手机。

---

## Slide 3: The Tale of Two Giants (双雄争霸：Android vs iOS)

- 页面视觉：
  - 一个天平或对比图。
  - **Android**: Robot Logo. Keywords: **Open (开放)**, **Diverse (多样)**, **Linux-based**.
  - **iOS**: Apple Logo. Keywords: **Walled Garden (围墙花园)**, **Integrated (整合)**, **Hybrid Kernel**.
- 核心内容:
  - 市场呈现双寡头垄断 ^^。
  - Android 基于 Linux 宏内核改良 ^^；iOS 基于 XNU 混合内核 ^^^^^^^。
  - +2
- 演讲要点 (Script Idea):

- "In the mobile world, we have two philosophies. The **Open** philosophy of Google's Android, and the **Closed** 'Walled Garden' of Apple's iOS."
- "Android creates a system for *everyone*—Samsung, Xiaomi, Pixel. iOS builds a system *only* for iPhone. This fundamental difference shapes their deep architecture, which we will see next."

---

## Slide 4: Android Architecture - The "Universal Translator" (深度: Android 架构)

- 页面视觉:
  - 画一个层级图 (简化版):
    1. **Apps** (Top)
    2. **ART (Android Runtime)**
    3. **HAL (Hardware Abstraction Layer)** <--- 高亮这个!
    4. **Linux Kernel** (Bottom)
- 核心内容 (必须讲透):
  - **HAL (硬件抽象层)**: 解决 GPL 协议问题, 允许厂商保护驱动源码 ^^。
  - **Binder**: Android 特有的高效 IPC 机制, 不同于 Linux 的管道 ^^。
- 演讲要点 (Script Idea):
  - "Let's look inside Android. Because Android needs to run on thousands of different chips, it uses a trick called the **HAL (Hardware Abstraction Layer)** ^^."
  - (这里用比喻): "Think of HAL as a **Universal Translator**. It allows the Android software to talk to any hardware—whether it's a Snapdragon or a MediaTek chip—without Google needing to know the manufacturer's trade secrets."
  - "Also, unlike standard Linux, Android uses **Binder** ^^ for communication, which is optimized for speed on low-memory devices."
- 注意点:
  - **不要照念**: 不要念 "Android uses Linux kernel... IPC mechanism..." 这种枯燥的句子。用 "Universal Translator" 这种比喻会让老师觉得你真的懂。

---

## Slide 5: iOS Architecture - The "Speed Demon" (深度: iOS 架构)

- 页面视觉：
  - 层级图：
    1. **Cocoa Touch** (UI Layer) <--- 高亮: **Highest Priority**
    2. **Media / Core Services**
    3. **Core OS (XNU Kernel)**
  - 关键词: **Sandboxing** (沙盒), **UI Priority**.
- 核心内容:
  - **XNU 内核**: 混合了 Mach 微内核和 BSD ^^。
  - **Sandboxing**: 每个应用独立容器, 无法直接访问其他应用 ^^。
  - **UI 渲染优先级**: 保证触摸体验流畅 ^^。
- 演讲要点 (Script Idea):
  - "On the other side, we have iOS. Apple controls both the hardware and software, so they don't need a heavy abstraction layer."
  - "Their secret weapon is **Optimization**. In iOS, the User Interface (UI) gets the highest priority^^. If you touch the screen, the system drops everything else to respond. That's why iPhones often feel 'smoother' even with less RAM."
  - "Also, for security, every app lives in a **Sandbox**^^. Instagram cannot look at your Banking App's data. They are totally isolated."

---

## Slide 6: The Price of Freedom (过渡: 碎片化挑战)

- 页面视觉:
  - 一张震撼的图: "Android Fragmentation" (网上有那种无数个不同尺寸方块组成的图)。
  - 数据: Android 15 已出, 但仍有 27% 的人用 Android 14, 大量人用旧版本 ^^。
  - 对比: iOS 的饼图, 绝大多数人都在最新版。
- 核心内容:
  - 碎片化是 Android 生态最大的挑战 ^^。
  - 解决办法: **Project Treble** (解耦) ^^。
- 演讲要点 (Script Idea):

- "But Android's openness comes with a cost: **Fragmentation**."
- "While almost all iPhone users are on the latest iOS, Android users are scattered across dozens of versions^^. This is a nightmare for developers."
- "Google is fixing this with **Project Treble**^^, which separates the core OS from the vendor's changes, making updates easier."
- **(转场 Transition):** "However, while Mobile OS struggles with *updates*, there is another type of OS that struggles with something even more critical: **Time**. For the next part, [Speaker 2 Name] will take us into the world of Embedded Systems."
- **注意点:**
  - **转场要自然:** 这里的转场词非常重要，能够体现团队合作的流畅度 (Fluency)。身体转向 Speaker 2，手势示意交接。

## Speaker 2: Embedded OS & Real-Time Constraints

### Slide 7: Defining the Invisible (定义：什么是嵌入式 OS?)

- **页面视觉:**
  - 左侧：一把瑞士军刀（代表 Mobile OS，什么都能做但都不精）。
  - 右侧：一把手术刀或精密钻头（代表 Embedded OS，只做一件事但做到极致）。
  - 文字：**Embedded OS: Dedicated, Reliable, Efficient**^^.
- **核心内容 (Key Points):**
  - 与通用 OS 不同，嵌入式 OS 是为特定硬件和特定任务而生 ^^。
  - 核心价值：**确定性 (Determinism)**、**可靠性 (Reliability)** ^^。
- **演讲要点 (Script Idea):**
  - "Thanks, [Speaker 1 Name]. While Mobile OS is like a 'Swiss Army Knife' designed to do everything, Embedded OS is like a 'Scalpel'—designed for **one specific task**."
  - "It runs inside your washing machine, your car's engine, or a medical robot. It doesn't need to play Angry Birds; it needs to run 24/7 without crashing for 10 years^^."
- **注意点 (High Score Tips):**
  - **对比强烈:** 用“瑞士军刀 vs 手术刀”的比喻，瞬间让非技术背景的同学也能听懂。

### Slide 8: The "Time" Concept - RTOS (核心概念：实时操作系统)

- **页面视觉:**

- 大标题: **RTOS (Real-Time Operating System)**.
- 图解: 一条时间轴, 标记着 "Deadline" (截止时间)。
- 对比:
  - **General OS**: "I'll do it when I can." (Best Effort).
  - **RTOS**: "I will do it at exactly 10:00:01." (Deterministic).
- 核心内容 (必须讲透):
  - RTOS 的核心特征是**保证任务在规定时间内完成** ^^。
  - 案例: 心脏起搏器或汽车安全气囊。延迟 = 生命危险 ^^。
- 演讲要点 (Script Idea):
  - "The most important type of Embedded OS is **RTOS**. What does 'Real-Time' mean?"
  - "It doesn't just mean 'fast'. It means **Deterministic**."
  - "Imagine an Airbag system. If you crash, the sensor sends a signal. A standard Windows or Android system might say: 'Wait, I'm busy updating an app, give me 0.5 seconds.' -> **The driver dies.**"
  - "An RTOS says: 'I will deploy the airbag in exactly 0.001 seconds, no matter what.' -> **The driver lives.** ^^"
- 注意点:
  - **关键词重音**: 读到 "Deterministic" 时一定要放慢语速, 重读。这是拿分的各种核心词汇。

## Slide 9: Under the Hood - Scheduling (深度: 调度策略)

- 页面视觉:
  - 两张简单的调度图对比:
    - **General OS (Linux CFS)**: 大家都排队, 公平竞争 (Fairness) ^^。
    - **RTOS (Preemptive)**: 高优先级任务直接插队, 踢走低优先级任务 (Preemptive) ^^。
- 核心内容:
  - RTOS 采用基于优先级的**抢占式调度 (Preemptive Priority Scheduling)** ^^。
  - 高优先级任务 (如传感器中断) 一旦就绪, 立即执行, 延迟在微秒( $\mu$ s)级别 ^^。
- 演讲要点 (Script Idea):

- "How does RTOS achieve this? Through **Preemptive Scheduling**."
- "In a General OS like Linux, the scheduler tries to be 'Fair' to all apps. But in RTOS, there is no fairness. There is only **Priority**."
- "If a 'High Priority' task (like the brakes) wakes up, the OS immediately **pauses** the 'Low Priority' task (like the radio). The switch happens in microseconds^^."
- 注意点:
  - 动作配合: 做一个“切断”的手势, 表示“Preemptive” (抢占) 的霸道。

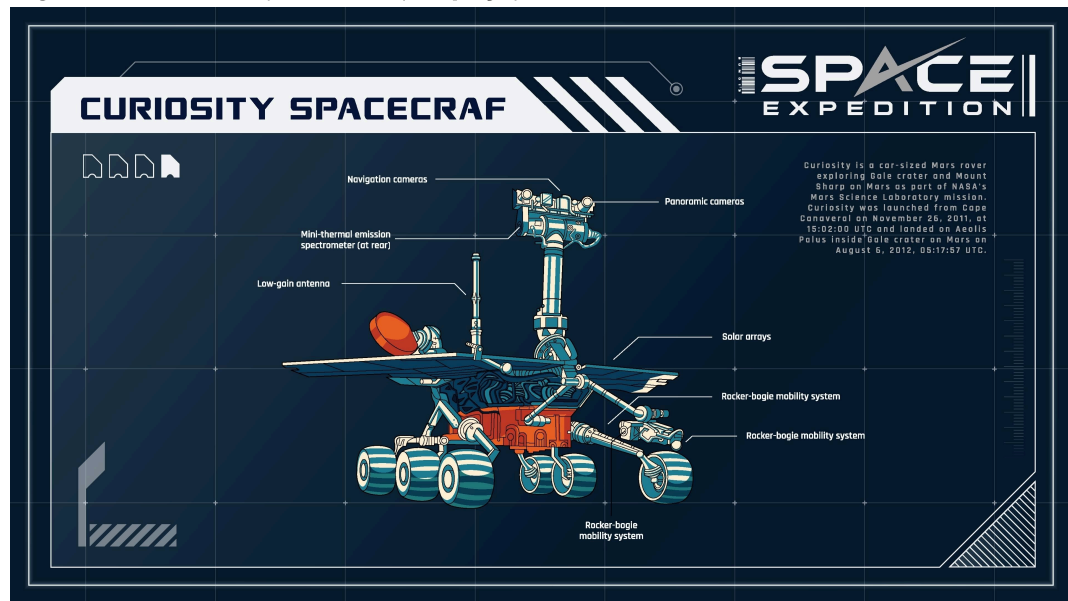
## Slide 10: Living on a Budget - Resource Constraints (深度: 资源与功耗)

- 页面视觉:
  - 对比图:
    - Desktop RAM: 16 GB.
    - MCU RAM: 16 KB.
  - 列出两个技术点:
    1. **No Dynamic Memory (Static Allocation)**^^.
    2. **Tickless Idle (Low Power)**^^.
- 核心内容:
  - 内存: 为了避免碎片化, RTOS 倾向于**静态内存分配**, 不用 `malloc` ^^。
  - 功耗: **Tickless Idle** 技术让 CPU 在空闲时进入深度睡眠, 而不是空转 ^^。
- 演讲要点 (Script Idea):
  - "Besides time, Embedded OS must manage extreme constraints. We don't have Gigabytes of RAM; we often have Kilobytes."
  - "So, we don't use dynamic memory allocation (like `malloc` ) because it's risky. We allocate everything **statically** at compile time to prevent crashes^^."
  - "For power, we use **Tickless Idle**. The CPU doesn't just wait; it goes into a deep sleep until the next interrupt, allowing devices to run on a coin battery for years^^."

## Slide 11: The Industry Standard (过渡: 主要玩家)

- 页面视觉:

- Logo 墙: FreeRTOS, VxWorks, Zephyr, Embedded Linux.



Shutterstock

(标注: Runs on VxWorks). \*



Getty Images

(标注: Runs on FreeRTOS/Zephyr).

- 核心内容:



- FreeRTOS: 极小内核, 适合微控制器 ^^。
- VxWorks: 火星探测器使用, 硬实时 ^^。
- Embedded Linux: 用于复杂网关 ^^。
- **演讲要点 (Script Idea):**
  - "Here are the legends of this field. **FreeRTOS** runs on billions of small IoT devices. **VxWorks** is so reliable it literally runs on Mars Rovers^^."
  - "But recently, the line between these simple embedded chips and powerful mobile phones is blurring. How do we combine them? [Speaker 3 Name] will tell us about the future."
- **注意点:**
  - **转场 Transition:** 提到“界限模糊 (Blurring lines)”, 为 Speaker 3 的“融合”做铺垫。

## Speaker 3: Convergence, Frontiers & Conclusion

### Slide 12: The Convergence - "A Computer on Wheels" (融合: 以特斯拉/现代电车为例)

- **页面视觉:**
  - (或者一般的现代电动车座舱图)。
  - 文字: **The Software Defined Vehicle (软件定义汽车)**。
  - 标注: Dashboard (Screen) vs. Chassis (Brakes/Motor)。
- **核心内容 (Key Points):**
  - 现代汽车 (如 Tesla 或搭载 Android Automotive 的车型) 是多操作系统共存的典型场景 ^^。
  - **冲突:** 我们需要手机一样的丰富应用 (地图、Spotify), 又需要嵌入式系统的绝对安全 (刹车、转向)。
- **演讲要点 (Script Idea):**
  - "Thanks, [Speaker 2 Name]. We used to think Mobile and Embedded were separate worlds. But look at a Tesla or any modern EV. What is it? It's basically a **Computer on Wheels**."
  - "Inside this car, we have a huge conflict. We want the screen to run Spotify and Google Maps like an Android phone. BUT, we also need the brakes to work with the reliability of a Mars Rover."

- "You cannot run the brakes on Android. If Android crashes, you can't stop the car. That is unacceptable."

## Slide 13: The Architecture Solution - Virtualization (深度：虚拟化架构)

- 页面视觉：\*
  - 绘制一个清晰的层级图：
    - **Bottom:** Hardware (Car CPU).
    - **Middle: Hypervisor (Virtualization Layer)** <--- 高亮这个核心词!
    - **Top Left: Android Automotive OS** (Guest 1) -> Runs Maps/Music^^.
    - **Top Right: QNX / RTOS** (Guest 2) -> Runs Brakes/ADAS^^.
    - 中间加一道红色的墙: **Isolation (隔离)**。
- 核心内容 (必须讲透):
  - **Hypervisor (管理程序):** 允许 QNX (实时系统) 和 Android (娱乐系统) 在同一硬件上运行 ^^。
  - **Isolation (隔离性):** 即使 Android 系统死机 (蓝屏/重启), QNX 负责的仪表盘和刹车系统依然独立运行, 不受影响 ^^。
- 演讲要点 (Script Idea):
  - "So, how do Tesla or Volvo solve this? The answer is **Virtualization**."
  - "They use a **Hypervisor**. It allows two different Operating Systems to run on the *same* chip at the *same* time, completely isolated."
  - "On one side, you have **Android** for your music. On the other side, you have an **RTOS (like QNX)** for the digital dashboard and safety systems."
  - "The magic is **Isolation**. If your Spotify crashes and the Android screen freezes, the Hypervisor ensures the RTOS keeps running. Your speedometer and brakes will *never* freeze. This is the ultimate convergence of User Experience and Safety."
- 注意点 (High Score Tips):
  - **强调安全性:** 这是一个很好的互动点。问台下: "Who wants their brakes to buffer while updating an app?" 大家笑了, 你的分数就拿到了。

## Slide 14: The Frontiers - TinyML & Matter (前沿与拓展)

- 页面视觉:
  - 分左右两块:

- 左边: **TinyML**. 图标: 一个微小的芯片上画着一个大脑。
- 右边: **Matter Protocol**. 图标: 不同品牌的家电 (Apple, Google, Alexa) 连在一起。
- 核心内容:
  - **TinyML**: AI 正在向边缘下沉。允许在资源极受限的 MCU 上运行机器学习模型 (如电机震动检测), 无需上传云端 ^^。
  - **Matter**: 解决了物联网协议碎片化 (Zigbee/Wi-Fi) 的问题, 统一了应用层标准 ^^。
- 演讲要点 (Script Idea):
  - "Beyond cars, what is the future? I want to highlight two trends."
  - "First is **TinyML**. We are now shrinking AI brains to fit onto tiny microcontrollers. Your toaster or industrial sensor can now 'think' and detect faults locally, without needing the Cloud."
  - "Second is **Matter**. For years, smart home devices couldn't talk to each other. Matter is the new language that finally connects Embedded OS devices from Apple, Google, and Amazon into one unified network."

## Slide 15: Conclusion & Takeaway (总结)

- 页面视觉:
  - 简单的总结列表 (Bullet points)。
  - 背景图: Mobile 和 Embedded 两个圆圈重叠 (Venn Diagram)。
- 核心内容:
  - 移动 OS 正在变得更模块化 (Project Mainline) 和安全 (Secure Enclave) ^^。
  - 嵌入式 OS 正在变得更互联 (Matter) 和智能 (TinyML) ^^。
  - 未来的架构师需要理解这种 **Trade-off (权衡)**: 生态 vs 控制 ^^。
- 演讲要点 (Script Idea):
  - "To conclude, we looked at Chapter 9 not just as a textbook definition, but as a spectrum."
  - "Mobile OS focuses on the **Human Experience** and Ecosystem. Embedded OS focuses on **Machine Control** and Determinism."
  - "As Computer Scientists, our job in the future isn't just to pick one. It's to understand the **Trade-offs**—to know when to use the flexibility of Android, and when to demand the reliability of an RTOS."

- "Thank you. We are now open for questions."
  - **注意点 (High Score Tips):**
    - **有力结尾:** 最后一句 "Trade-offs" (权衡) 是计算机科学的核心思想, 用这个词结尾会显得非常有学术高度。
- 

## Speaker 3 需要特别注意的“坑”

1. **关于 Tesla 的准确性:** 虽然大家都知道 Tesla 用 Linux, 但 PPT 里讲原理时, 建议用 **"Modern EV Architecture (e.g., Android Automotive + QNX)"** 这种说法更严谨, 因为这是行业通用的方案 (文档^^也特指了 QNX/Android 组合)。口头举例可以说 "Like Tesla", 但 PPT 图示最好展示通用的 Hypervisor 架构。
2. **TinyML 别讲太细:** 只要解释“在微小芯片上跑 AI”就行了, 不要去解释 TensorFlow Lite 的量化过程, 那是另一个课题了。
3. **Q&A 准备:** 老师可能会问: “为什么不能只用一个超强的 Linux 内核解决所有问题?”
  - **标准答案:** "Because standard Linux is not **Real-Time**. It cannot guarantee microsecond latency for safety-critical tasks like braking." (引用 Speaker 2 的观点, 体现团队一致性)。

这样, 你们的三人 PPT 大纲就全部完成了。逻辑链条是:

1. **Speaker 1:** 为什么要有非传统 OS? (手机架构与碎片化)
2. **Speaker 2:** 什么是嵌入式 OS? (实时性与保命)
3. **Speaker 3:** 它们如何在一起工作? (汽车虚拟化与未来)

祝你们 Presentation 顺利! 需要我把这些整合成一个完整的文本文件或者 Markdown 供你下载吗?